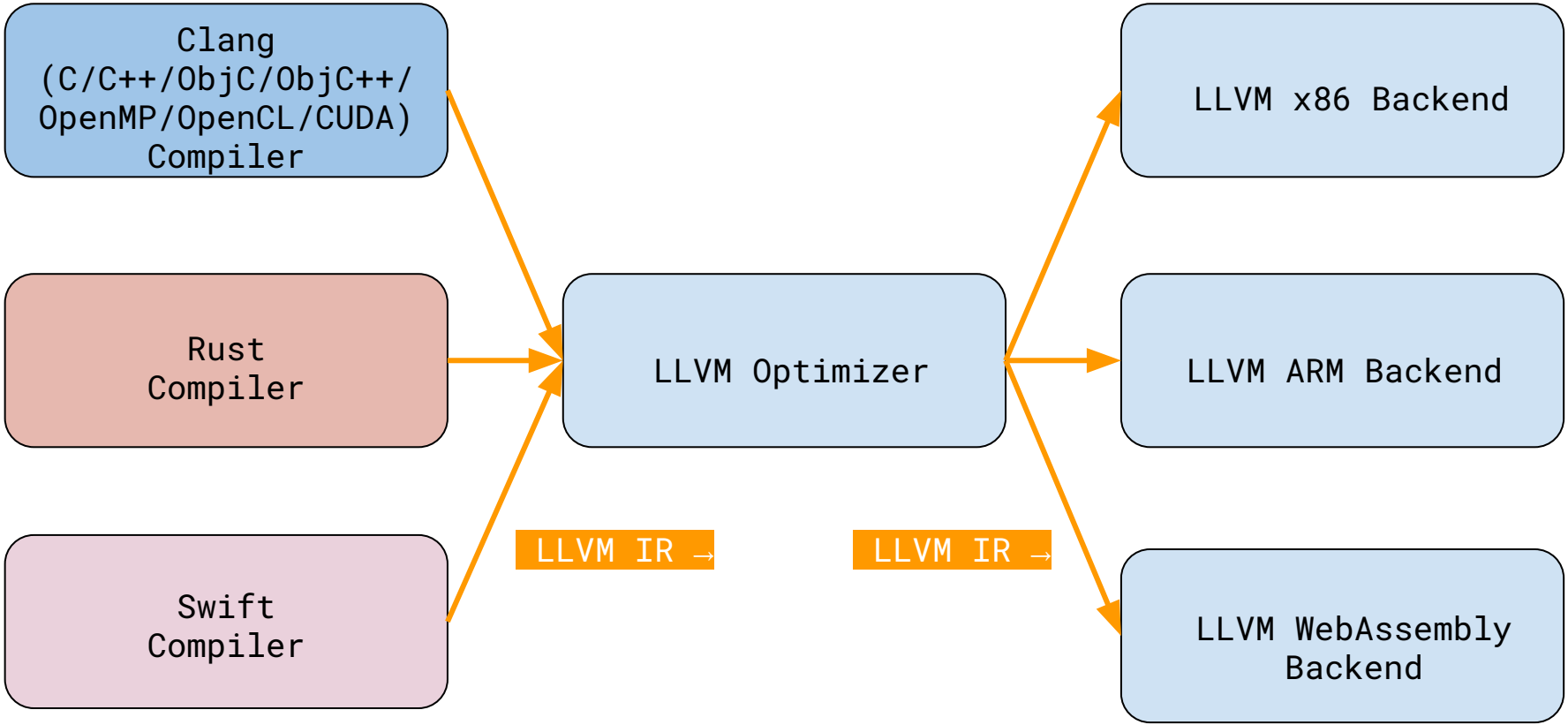
Google

Compiling the Linux Kernel with Clang

Linux Plumbers Conf

14 Nov 2018

What is Clang and LLVM?



A fix in one part of LLVM benefits all

…

…

Google

Alternatives to GCC and Binutils

clang	gcc
clang++	g++
clang (“integrated assembler”)	as
ld.lld	ld.bfd , ld.gold
llvm-objdump, llvm-objcopy, llvm-*, etc	objdump, objcopy, strip, readelf, etc
Compiler_RT	libgcc
libc++	libstdc++

Google

Proprietary + Confidential

Why build the kernel with Clang?

- New features built on compiler tech
  - LTO, autoFDO (requires ETM on ARM), CFI, SCS, BOLT, clangd, lld
- Static and Dynamic Analyses
  - Static Analyzer, Thread Safety Analysis, KASAN, KHWASAN, KUBSAN, Syzkaller
- Additional warning flag coverage
- Shake out undefined behavior
- Improve both code bases
- One binary for cross compiling (`-target <triple>` flags, rather than one toolchain per arch)
- Optionality in toolchains

Proprietary + Confidential

Priorities

1. Get it building
2. Get it booting
3. Continuous Integration
4. Get it building warning free
5. Polish
6. Add features

1. arm64
2. arm
3. X86_64
4. PowerPC
5. RISC-V

1. Compile with Clang
2. Link with LLD
3. Assemble with Clang

Proprietary + Confidential

Recent Clang features for kernel

- -fno-delete-null-pointer-checks (landed in Clang 7)
- rN register naming for aarch64 (landed in Clang 7)
- -fcall-used -fcall-saved for aarch64 (landed in Clang 8)
- asm goto (prototype LLVM patch from Intel: https://reviews.llvm.org/D53765)
- (more fixed issues in Clang)

Proprietary + Confidential

Give it a shot

$ make CC=clang HOSTCC=clang

Example cross compiling (arm64/aarch64):

$ ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- make CC=clang HOSTCC=clang

Link with LLD

$ make LD=ld.lld

Note: LLVM moves quickly! If your distro's clang doesn't work, try the latest release at https://releases.llvm.org/ or from the AOSP prebuilts/clang/host/linux-x86 project.

Google

Proprietary + Confidential

Distros shipping Clang built kernels

- Android
  - Pixel 2 - Oct 2017 (Clang)
  - Pixel 3 - Oct 2018 (Clang+LTO+CFI)
  - Expect a LOT more Android devices, soon.
- CrOS
  - New devices with 4.19, 4.14, and 4.4 based kernels.

Continuous Automated Testing

- Linaro's KernelCI is ramping up support for Clang.
  - WIP driving warnings to zero on kernel and Clang side.
  - allyesconfig not "all" code in kernel; randconfig? Blessed configs?
  - -next vs mainline vs LTS?
- In communication with Intel's 0day bot team.
- Setting up internal and external qemu+buildroot based boot testing.
  - Daily clang snapshots, build linux with clang, boot buildroot in qemu, run tests.
  - Qemu scales better than physical devices.
  - Docker seems to simplify dependency management?

Report bugs

- https://github.com/ClangBuiltLinux/linux/issues
  - We try to flag/save low hanging fruit for folks looking to get started contributing to Linux, see `good-first-issue` tag.
  - File a blocker: https://bugs.llvm.org/show_bug.cgi?id=4068 cc: ndesaulniers@google.com
- godbolt.org: Great for sharing reproducers
- creduce: Minimize reproducers
- bear: compile_commands.json for compiler flags
- .<target>.o.cmd files produces by a build contain the exact compiler flags

Problematic code for Clang

- VLAiS (Gone! Kees Cook removed them, added -Wvla)
- Nested functions (not used in kernel)
- __builtin_constant_p (has many edge cases, 2 final patches for Clang to match more of GCC)
- asm goto (patch set in progress, testing on the kernel ongoing)
- A few warning false negatives in Clang (we have them documented, working through fixing them)
- Code that is only semantically valid in GCC at -O2 (some static __attribute__((always_inline))

  functions with extended inline asm constraints that don't make sense until AFTER inlining.

Thanks

Thanks to the many people making this all work.

We’re proud to present this work on their behalf and lucky to have such interest and support.

If you’re interested in how you can help, we want to enable you to be successful, get in touch!